COMP2411 Database Systems

The Hong Kong Polytechnic University

Farmlander Farm CSA Subscription Database

Stage 1 Proposal and Database Design

Group ID	G085
Canvas group name	Group 85, Nov 25 Wed 12:40-12:45 PQ604A
Scenario ID	D17-W04 (Local farm subscription and distribution + Membership or subscription)
Team size	2
Instructor	HUANG Xiao
Submission date	26 October 2026

Member name	Student ID
CHEN Guanda	25095282d
SHI Jingxuan	25101061d

Contents

1 Scenario Fingerprint and Scope **1**
1.1 Scenario Fingerprint . . . 1

2 Requirements **2**
2.1 Business Rules . . . 2
2.2 Operational Use Cases . . . 4
2.3 Management Questions . . . 5

3 Entity–Relationship Model **6**
3.1 ER Diagram . . . 7
3.2 Design Constructs and Justification . . . 7

4 Relational Schema and Mapping **8**
4.1 Relational Schema . . . 8

5 Data Dictionary **8**
5.1 `customer` . . . 9
5.2 `season` . . . 9
5.3 `subscription` . . . 9
5.4 `delivery` . . . 10
5.5 `refund` . . . 10
5.6 `product` . . . 11
5.7 `chosen_products` . . . 11
5.8 `add_on` . . . 11

6 Functional Dependencies and Normalization **12**
6.1 Analysis per Relation . . . 12

7 Traceability Matrix **15**

8 Work Plan and Risks **17**
8.1 Division of Work . . . 17
8.2 Implementation Risks . . . 17

References **19**

1 Scenario Fingerprint and Scope

The farmer lander follows a business pattern of weekly/biweekly delivery of farm products to subscribed customers[1]. In this group project, the database system designed for such business scenario mainly focuses on subscription management, including registration, renewal, and cancelation of the subscription for weekly/biweekly agriculture products delivery[4].

1.1 Scenario Fingerprint

Table 1. Scenario fingerprint (template of Appendix B of the project guidance)

Group ID	G085
Full Canvas group name	Group 85, Nov 25 Wed 12:40-12:45 PQ604A
Scenario ID	D17-W04
Organization name and type	Farmerland - Agriculture Cooperative
Principal users/roles	Farm Staff, Subscriber
Business object and core event	• Business object: a seasonal CSA subscription with a fixed 12-week delivery window • Core event: enrolment, cancelation, and renewal for subscription; skipping and making add-on purchase on each delivery; recording each delivery's outcome; calculating subscription or refund fee based on business rules[4]
Distinctive policy or constraint	• Subscriptions are seasonal and only operate in spring, summer, and autumn. • The enrolment window is the previous season (winter -> spring, spring -> summer, summer -> autumn)[4] • A renewal gets 20% off the allocated cost per delivery • Plan type and frequency are fixed at enrolement[5] • Each season farmers grow exactly 10 types of agriculture products • Available types of products in add-on purchase for each delivery are limited to the plan's products
Management questions (at least four)	Q1. For the Spring 2026 season, how many subscriptions were sold per plan type, and what is the total refund amount (skips, failed deliveries, cancellations) of each plan and net income after all refunds? Q2. For the Spring 2026, per plan type, which add-on products generate the largest add-on income from successful deliveries? Q3. Which delivery weeks of Spring 2026 had the highest skip rates and the highest failure rates? Q4. For the Spring 2026 season, which products are most frequently chosen by customizable plan subscribers, and are they among the season's 6 farmer's choice products? Q5. For subscriptions in the Spring 2026 season that were not cancelled, what proportion of each plan type's subscribers renewed into Summer 2026, and which plan type had a renewal rate below 60%?
Excluded processes	No real payment processing; No delivery assignment; No farm production management beyond the given types

Table 1 (continued)

Workflow fit and scope rationale	• Workflow fit: In the CSA workflow, customers receive weekly or biweekly deliveries of agricultural products[1]. They may subscribe for the next season, cancel subscriptions, skip deliveries, or purchase add-on products[5]. Therefore, the database design includes entities for customer, subscription, delivery, product, and add-on Purchase. The remaining refund skipping time is stored and tracked because it represents a business quota that determines a customer's entitlement to fully compensated delivery skips. These entities and constraints directly support the operational workflow of CSA delivery management. • Scope rational: The scope focuses on the subscription management and customers' quota usage.
Difficulty rationale	The project involves membership management, subscription renewal, delivery recordings and detailed refunding rules, which satisfies the project complexity requirements while keeping the overall scope manageable.

2 Requirements

2.1 Business Rules

BR1 Subscription ownership

Rule	Every subscription is held by exactly one customer and is for exactly one season. A customer may hold zero or more subscriptions, and a season may have zero or more subscriptions. Each customer has a unique email.
Type	Cardinality, Participation

BR2 One subscription per customer per season

Rule	A customer can hold at most one subscription for the same season. A cancelled subscription still counts, which means its holder can not subscribe to that season again.
Type	Uniqueness

BR3 Season calendar and subscription status

Rule	A season for subscription is the spring, summer or autumn of one year. Winter is not offered as a season for subscription, but still treated as a seasonal time period for scheduling enrolment and renewal activities. Each subscribable season has 12 delivery planning days, the Monday of 12 consecutive weeks. When D is defined as the first planning delivery day, the subscription range runs from $D - 6$ to the 12th planning day $D + 77$, both included. The status of a subscription is planned before its range start, active during the range, finished after the range ends, and cancelled once the customer decides to cancel it. No other status is allowed.
Type	Value, Validity

BR4 Enrolment window and renewal

Rule	A subscription can only be submitted inside the window of the season before it (the previous year's winter for spring, spring for summer, summer for autumn). A subscription is renewal if, when it is submitted, the customer holds a subscription for the previous subscribable season whose status is active or finished. A renewal gets 20% off for its allocated cost per delivery.
Type	Eligibility

BR5 Plan, frequency and subscription fee

Rule	There are two types of subscription, including farmer's choice and customizable. The frequency of delivery for each plan is also optional, including weekly or biweekly. Neither the plan type nor the frequency can be changed once the subscription is submitted. To simplify calculation, the allocated cost per delivery is 200 for farmer's choice, 240 for customizable, or 160 and 192 for a renewal. The subscription fee is the number of deliveries (12 for weekly, 6 for biweekly) multiplies the allocated cost per delivery.
Type	Value

BR6 Product types of a plan

Rule	Each season has 10 distinctive product types and each belongs to that season only. 6 of them are flagged as the farmer's choice types. A farmer's choice subscription gets those 6 types and the customer does not choose them, so it has no chosen types. A customizable subscription must choose exactly 8 different types from its own season's 10 when it is submitted, and the choice is immutable once its range starts.
Type	Cardinality, Value

BR7 Delivery schedule and status

Rule	Each delivery belongs to exactly one subscription and falls on one planning day of that subscription's season, identified by its week number (1 to 12). A subscription has at most one delivery per planning day. On the first day of the range, each subscription that is not cancelled gets all its deliveries with status planned: weekly on all 12 planning days, biweekly on the 1st, 3rd, 5th, 7th, 9th and 11th only. A delivery's status can be planned, skipped, success, failed or cancelled. The only allowed changes are planned to skipped by the customer, planned to success or failed, recorded on the day after the delivery day, and planned to cancelled when the subscription is cancelled before the delivery day. A failed delivery refunds its price, that is, the subscription's allocated cost per delivery plus that delivery's add-on fee. The refund is recorded when the delivery is changed to failed and does not use a refundable skip.
Type	Cardinality, Participation, Uniqueness, Value

BR8 Skipping and refundable skips

Rule	A customer may skip a delivery only when its status is planned and only inside its change window, which is from the day after the subscription's previous delivery day to the day before this delivery day. For the first delivery, from 6 days before it (the first day of the range) to the day before, both included. Each subscription starts with 2 refundable skips. While at least one remains, a skip refunds the subscription's allocated cost per delivery plus that delivery's add-on fee, and uses up one refundable skip. When none remain, the delivery can still be skipped, but nothing is refunded.
Type	Validity, Quota

BR9 Add-on purchases

Rule	An add-on can only be bought for a delivery whose status is planned, inside the same change window as skipping in BR8. Its product types are limited to the subscription's plan. For farmer's choice, they are the season's 6 fixed types, or the 8 types the customer chose for a customizable subscription. Each product type appears at most once in a delivery's add-ons, with a weight of at least 0.1 kg. A delivery's add-on fee is the sum of weight multiplies that product's price per kg. The price of the delivery is its allocated cost per delivery plus this fee.
Type	Value, Validity, Uniqueness

BR10 Cancellation and refund

Rule	Only a planned or active subscription can be cancelled. If it is planned, which means its range has not started, the full subscription fee charged is refunded. If it is active, the refund is min(remaining refundable skips, number of planned deliveries whose delivery day is after the cancellation date) multiplies the subscription's allocated cost per delivery, plus the add-on fees on those planned deliveries. In both cases the refund is calculated first; then the remaining refundable skips are set to 0, those deliveries become cancelled, and the subscription becomes cancelled. A planned delivery on or before the cancellation date keeps waiting for its result (BR7). A finished subscription cannot be cancelled.
Type	Validity, Quota

BR11 Refund records

Rule	Every refund is for exactly one subscription and a subscription may have zero or more refunds. A refund's type is skip, failed or cancellation. The skip or failed refund is for exactly one delivery of the same subscription and a delivery has at most one refund. The cancellation refund is for no delivery.
Type	Cardinality, Participation, Value

2.2 Operational Use Cases

There are four user cases, 2 focused on subscription management and 2 focused on recording and processing of delivery.

UC1 Book a subscription

Description	When the date is inside the window for subscription of upcoming season, a customer asks to subscribes to the season.
Related operations	Creat, Retrieve

UC2 Record a delivery

Description	On the day after a scheduled Monday delivery, the farm staff record status of a delivery on that day.
Related operations	Creat, Retrieve, Change

UC3 Skip a delivery

Description	For a specific upcoming delivery, a customer who owns it asks to skip it.
Related operations	Creat, Retrieve, Change

UC4 Cancel a subscription with two refund cases

Description	A customer asks to cancel his/her subscription.
Related operations	Creat, Retrieve, Change

Table 2. Data operations performed by each use case (✓ = performed)

UC	Title	Create	Change	Retrieve
UC1	Book a subscription for an upcoming season	✓		✓
UC2	Record a weekly delivery	✓	✓	✓
UC3	Skip a delivery	✓	✓	✓
UC4	Cancel a subscription with two refund cases	✓	✓	✓

2.3 Management Questions

MQ1 Sales, refunds and net income per plan type

Question	For the Spring 2026 season, how many subscriptions were sold per plan type, and what is the total refund amount (skips, failed deliveries, cancellations) of each plan and net income after all refunds?
Tables	season, subscription, add_on, product, refund
Analysis	Join subscription to season and filter to season_name = 'spring' and year = 2026. For each subscription, compute its subscription fee, add-on income over its add_on rows joined to and its refund total in seperate derived table. Group by plan_type: subscription count = COUNT(*), total refund = SUM(refund total), net income = SUM(subscription fee + add-on income − refund total).
Decision supported	Quantify how much the refund policies actually cost per plan type in a season, informing whether the number of refundable skips should be reviewed.

MQ2 Top add-on products per plan type

Question	For the Spring 2026, per plan type, which add-on products generate the largest add-on income from successful deliveries?
Tables	season, subscription, delivery, add_on, product
Analysis	Join the subscription, season, delivery, add_on, product, filtering the season to Spring 2026, keep only deliveries with status = 'success', GROUP BY plan_type and product_id, and compute income as SUM(weight x price_per_kg) and total weight as SUM(weight). Order products by income within each plan type.
Decision supported	Identify the most demanded add-on products per plan type and help to decide which products farmers should grow in larger quantities for add-on sales

MQ3 Skip and failure rates per delivery week

Question	Which delivery weeks of Spring 2026 had the highest skip rates and the highest failure rates?
Tables	season, subscription, delivery
Analysis	Join delivery to the subscription and season, filter to Spring 2026 and exclude cancelled deliveries. GROUP BY week_no and use conditional aggregation on skips = SUM(status = 'skipped'), failures = SUM(status = 'failed'), attempted = SUM(status IN ('success', 'failed')). Skip rate = skips ÷ COUNT(*) of the week. Failure rate = failures ÷ attempted (NULLIF avoids division by zero).

Decision supported	Informing the week with highest skipping rate helps with planning future delivery and harvest volumes. Weeks with high failure rates flag potential supply or staff problems. They do not explain why deliveries failed or why customers skipped.

MQ4 Most chosen products of the customizable plan

Question	For the Spring 2026 season, which products are most frequently chosen by customizable plan subscribers, and are they among the season's 6 farmer's choice products?
Tables	season, subscription, chosen$_p roducts, product$
Analysis	Product joins the season filter to the Spring 2026. LEFT JOIN chosen_products whose product_id in the filtered Spring 2026 list with subscription. GROUP BY the name and is_farmers_choice of products, count how many subscriptions chose it with COUNT(chosen_products.subscription_id) and compute the rate as that count / the number of customizable subscriptions of the season. Rank the count with ORDER BY.
Decision supported	Use actual customer preferences to review which 6 products are flagged as the farmer's choice box for the next season.

MQ5 Renewal rate per plan type

Question	For subscriptions in the Spring 2026 season that were not cancelled, what proportion of each plan type's subscribers renewed into Summer 2026, and which plan type had a renewal rate below 60%?
Tables	season, subscription
Analysis	Join subscription to season and filter to Spring 2026 with status to be not 'cancelled'. LEFT JOIN subscription again with season filtered to Summer 2026 on the same customer_id, so customers who did not renew will be null. At most one match per customer because of UNIQUE (customer_id, season_id). GROUP BY the Spring 2026 plan_type. Renewal rate = COUNT(summer subscription_id) / COUNT(spring, subscription_id), since COUNT will ignore the nulls, HAVING renewal rate < 0.60
Decision supported	Flag plan types with a renewal rate below 60% for a review of their renewal offer or product mix before the next subscription window opens. A low rate in synthetic data flags where to look; it does not explain why customers did not renew.

Table 3. Coverage of the management-question requirements

MQ	Aggregate	Tables used	3+ tables	Planned SQL techniques (Stage 2)
MQ1	✓	season, subscription, add_on, product, refund	✓	JOIN, GROUP BY, COALESCE, SUM, COUNT
MQ2	✓	season, subscription, add_on, product, refund	✓	JOIN, GROUP BY, SUM
MQ3	✓	season, subscription, delivery	✓	JOIN, GROUP BY, SUM
MQ4	✓	season, subscription, chosen_products, product	✓	JOIN, LEFT JOIN, GROUP BY, COUNT
MQ5	✓	season, subscription		JOIN, LEFT JOIN, GROUP BY, COUNT

3 Entity–Relationship Model

3.1 ER Diagram

The subscription serves as a core entity in the relational diagram, which connects with all other entities. Delivery is a weak entity whose owner entity is subscription. There is a 'belong to' relation between the subscription and the customer whose address attribute is composite. The add on relation between product and delivery has a weight attribute that indicates how much the product weighs in the add on part of delivery. In addition, there are several derived attributes in this diagram, such as fee() in subscription, start_day() in season, and delivery_day in delivery.

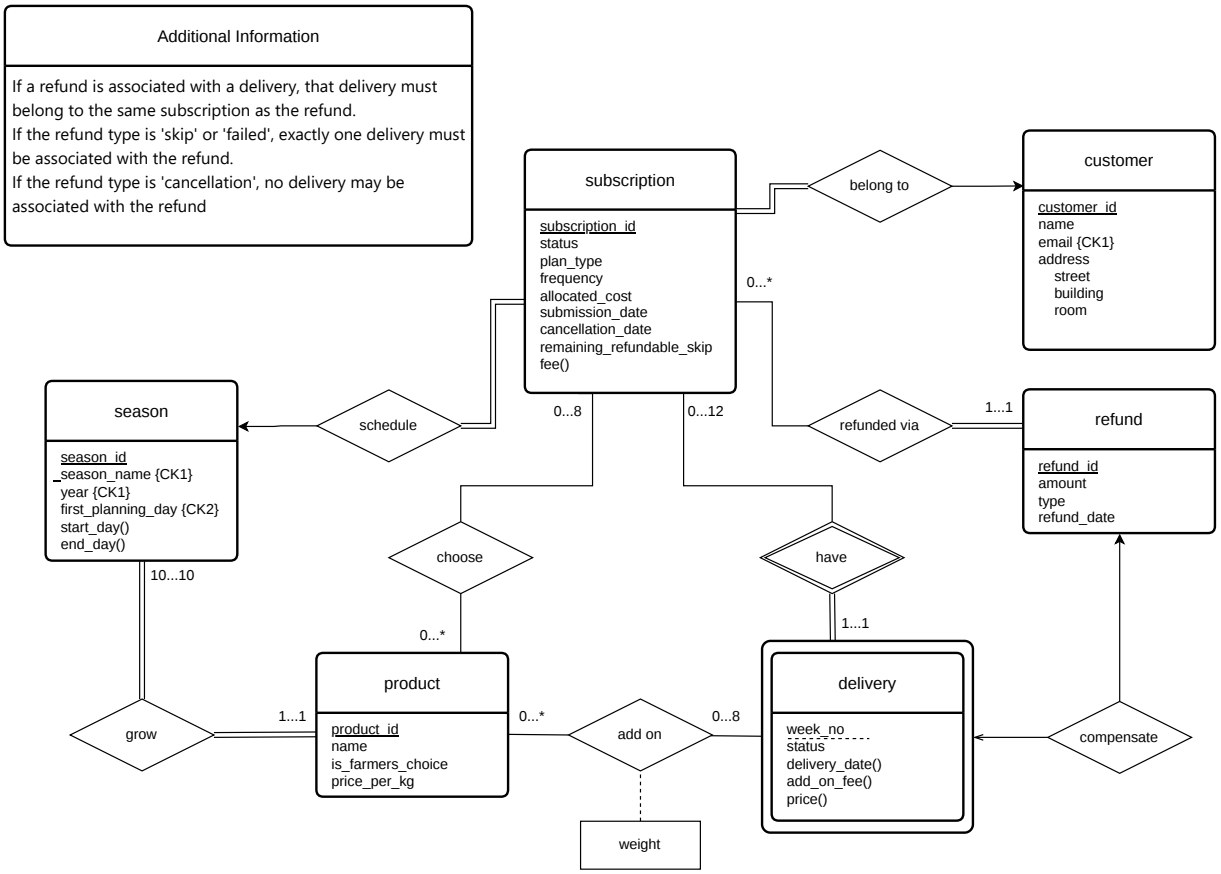


Figure 1. ER diagram of the Farmlander Farm CSA subscription database

3.2 Design Constructs and Justification

Table 4. ER constructs used and their justification

Construct	Where used	Justification
Weak entity	delivery	The delivery identity cannot be uniquely identified by week_no without the subscription_id
Composite attribute	customer	The address of customer can be meaningfully decomposed into street, building, and room, which have independent business significance.

Table 4 (continued)

Construct	Where used	Justification
Derived attributes	subscription, season, delivery	For a season, the planning delivery day in each week is fixed to Monday. Given the first_planning_day and the rules on the range of a season in BR3, the start_day and end_day can be derived. Similarly to the season, given a week_no and subscription_id, the delivery_date can also be determined. Once the subscription_id and the weight of each add-on product are known, both the add_on_fee and the price can be calculated. Given the allocated_cost and frequency of a subscription, its fee can be calculated by multiplying the number of planned deliveries by the allocated_cost.
M:N relationship with attribute	add on	Each delivery may include add-on products of different types and weights. Therefore, weight is modeled as an attribute of the add-on relationship. Since weight is only used to determine pricing and does not have an independent lifecycle, the add-on relationship does not need to be modeled as a separate entity.

4 Relational Schema and Mapping

4.1 Relational Schema

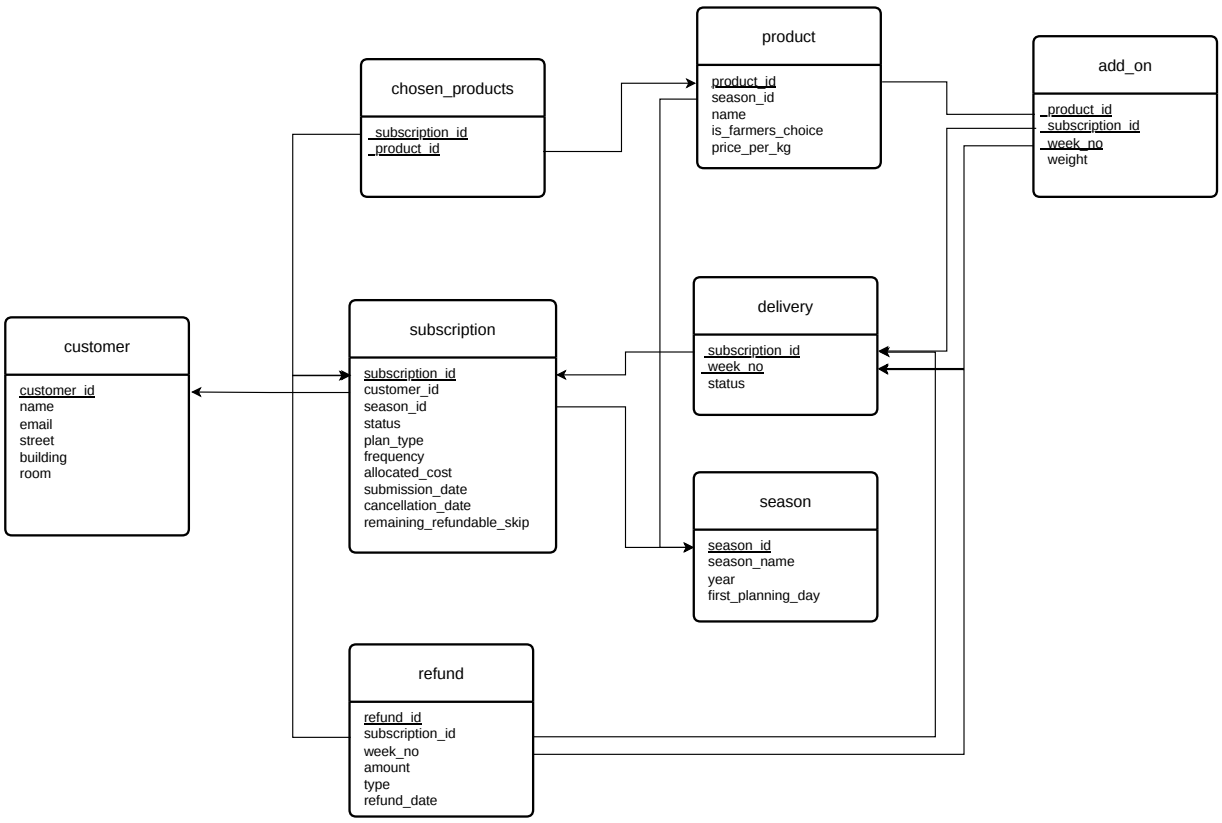


Figure 2. Relational schema

5 Data Dictionary

5.1 customer

Table 5. Data dictionary of customer

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
customer_id	INT UNSIGNED AUTO_INCREMENT	PK	NO	–	–	Surrogate identifier
name	VARCHAR(100)	–	NO	–	–	Full name
email	VARCHAR(255)	UNIQUE (CK1)	NO	–	–	Login/contact; candidate key
street	VARCHAR(255)	–	NO	–	–	Address part 1 (composite attr)
building	VARCHAR(50)	–	NO	–	–	Address part 2
room	VARCHAR(20)	–	NO	–	–	Address part 3

5.2 season

Table 6. Data dictionary of season

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
season_id	SMALLINT UNSIGNED AUTO_INCREMENT	PK	NO	–	–	Surrogate identifier
season_name	VARCHAR(10)	part of CK1	NO	–	IN ('spring', 'summer', 'autumn')	Season within a year
year	SMALLINT UNSIGNED	part of CK1	NO	–	–	Calendar year
first_planning_day	DATE	UNIQUE (CK2), CHECK (DAYOFWEEK(first_planning_day) = 2)	NO	–	–	Anchor date; start_day() $= -6$, end_day() $= +77$ (derived, not stored)

5.3 subscription

Table 7. Data dictionary of subscription

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
subscription_id	INT UNSIGNED AUTO_INCREMENT	PK	NO	–	–	Surrogate identifier
customer_id	INT UNSIGNED	FK → customer; part of CK	NO	–	UNIQUE(customer_id, season_id)	Holder (BR1: at most 1 per customer per season; cancelled still occupies the slot)
season_id	SMALLINT UNSIGNED	FK → season; part of CK	NO	–	–	Season subscribed
status	VARCHAR(10)	–	NO	'planned'	CHECK IN ('planned', 'active', 'finished', 'cancelled')	Lifecycle state
plan_type	VARCHAR(20)	–	NO	–	CHECK IN ('farmer''s choice', 'customizable')	Chosen at enrolment, immutable afterwards

Continued on next page

Table 7 (continued)

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
frequency	VARCHAR(10)	–	NO	–	CHECK IN ('weekly', 'biweekly')	Chosen at enrolment, immutable afterwards
allocated_cost	DECIMAL(8,2)	–	NO	–	CHECK IN (160, 192, 200, 240)	Single-delivery price snapshot incl. 20% renewal discount (stored to lock price)
submission_date	DATE	–	NO	–	–	Enrolment/renewal date
cancellation_date	DATE	–	YES	NULL	–	NULL = not cancelled (legitimate missing value)
remaining_refundable_skip	TINYINT	–	NO	2	CHECK BETWEEN 0 AND 2	Refundable skip quota, consumed by skip refunds

Derived: fee() = delivery count × allocated_cost; see Conventions above.

5.4 delivery

Table 8. Data dictionary of delivery

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
subscription_id	INT UNSIGNED	PK part 1; FK → subscription	NO	–	–	Owner entity PK (identifying relationship)
week_no	TINYINT UNSIGNED	PK part 2 (partial key)	NO	–	CHECK (BETWEEN 1 AND 12)	Week within the season
status	VARCHAR(10)	–	NO	'planned'	CHECK IN ('planned', 'skipped', 'success', 'failed', 'cancelled')	Delivery state machine; transitions guarded by UPDATE in transactions

Derived: delivery_date() = first planning delivery day + (week_no − 1) × 7; add_on_fee() = SUM(weight × price_per_kg); price() is computed on the fly. See Conventions above.

5.5 refund

Table 9. Data dictionary of refund

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
refund_id	INT UNSIGNED AUTO_INCREMENT	PK	NO	–	–	Surrogate identifier
subscription_id	INT UNSIGNED	FK → subscription	NO	–	–	The subscription refunded
amount	DECIMAL(8,2)	–	NO	–	CHECK (> 0)	Computed at discounted allocated cost (anti-arbitrage)
type	VARCHAR(12)	–	NO	–	CHECK IN ('skip', 'failed', 'cancellation')	'skip' consumes quota; 'failed' does not; 'cancellation' ties to no delivery

Continued on next page

Table 9 (continued)

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
refund_date	DATE	–	NO	–	–	When the refund was issued
week_no	TINYINT UNSIGNED	Make up the composite foreign key with subscription_id referencing delivery(subscription_id, week_no); UNIQUE(subscription_id, week_no)	YES	NULL	CHECK: If type is skip or failed, it should be non null; if type is cancellation, it must be null	Null indicates cancellation, which does not refer to any delivery; When it is non null, the composite foreign key guarantees the refund and delivery belong to the same subscription.

5.6 product

Table 10. Data dictionary of product

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
product_id	INT UNSIGNED AUTO_INCREMENT	PK	NO	–	–	Surrogate identifier
season_id	SMALLINT UNSIGNED	FK → season	NO	–	UNIQUE(season_id, name)	The season it is grown in (exactly 10 products per season; cross-row rule → in-transaction check)
name	VARCHAR(100)	–	NO	–	UNIQUE(season_id, name)	Product name
is_farmers_choice	TINYINT(1)	–	NO	0	–	1 = one of the 6 fixed farmer's-choice items
price_per_kg	DECIMAL(6,2)	–	NO	–	CHECK (>= 0)	Unit price for add-on charging

5.7 chosen_products

Table 11. Data dictionary of chosen_products

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
subscription_id	INT UNSIGNED	PK part 1; FK → subscription	NO	–	–	Customizable plan's picks
product_id	INT UNSIGNED	PK part 2; FK → product	NO	–	–	Picked product (must be grown in the same season; cross-table → in-transaction check)

Note: "0..8 picks" is a cross-row rule and cannot be a CHECK; enforced by in-transaction COUNT(*) check.

5.8 add_on

Table 12. Data dictionary of `add_on`

Column	MySQL type	Key	Null	Default	Constraint / domain	Description
`subscription_id`	`INT UNSIGNED`	PK part 1; composite FK → `delivery`	NO	–	–	Delivery the add-on belongs to
`week_no`	`TINYINT UNSIGNED`	PK part 2; composite FK → `delivery`	NO	–	–	Same window as refundable skip
`product_id`	`INT UNSIGNED`	PK part 3; FK → `product`	NO	–	–	Product added (limited to the plan's types)
`weight`	`DECIMAL(5,2)`	–	NO	–	`CHECK (>= 0.10)`	Relationship attribute from ER diagram; fee = SUM(`weight` × `price_per_kg`)

Note: (`subscription_id`, `week_no`) together form ONE composite foreign key referencing `delivery(subscription_id, week_no)`.

6 Functional Dependencies and Normalization

6.1 Analysis per Relation

`customer`

Functional dependencies
- `customer_id` → `name`
- `customer_id` → `email`
- `customer_id` → `street, building, room`
- `email` → `customer_id`
- `email` → `name`
- `email` → `street, building, room`

Candidate keys
- `customer_id`
- `email`

Normal form
Satisfies BCNF, because the left-hand side of every non-trivial functional dependency {`customer_id` or `email`} is a candidate key, and hence a super key. The address {`street`, `building`, `room`} and `name` do not determine any other attribute, since several customers may share the same address or name.

Lossless join and dependency preservation
Lossless: No decomposition needed, not applicable.
Preserving: No decomposition needed, not applicable.

`season`

Functional dependencies
- `season_id` → `season_name, year, first_planning_day`
- `season_name, year` → `season_id, first_planning_day`
- `first_planning_day` → `season_id, season_name, year`

Candidate keys
- `season_id`
- `season_name, year`
- `first_planning_day`

Normal form
Every determinant (`season_id` / {`season_name, year`} / `first_planning_day`) is itself a candidate key, hence a superkey. `season` is in BCNF.

Lossless join and dependency preservation	Lossless: No decomposition needed, not applicable. Preserving: No decomposition needed, not applicable.

`subscription`

Functional dependencies	• `subscription_id` → `customer_id` • `subscription_id` → `season_id` • `subscription_id` → `status` • `subscription_id` → `plan_type, frequency` • `subscription_id` → `allocated_cost` • `subscription_id` → `submission_date, cancellation_date` • `subscription_id` → `remaining_refundable_skip` • `submission_date` → `season_id` • `customer_id, season_id` → `subscription_id, status, plan_type, frequency, allocated_cost, submission_date, cancellation_date, remaining_refundable_skip`
Candidate keys	• `subscription_id` • `customer_id, season_id` • `customer_id, submission_date`
Normal form	Satisfies 3NF but not BCNF, because the functional dependency of `submission_date` → `season_id` is not a trivial dependency and `submission_date` is not a super key. However, {`season_id`} − {`submission_date`} = {`season_id`} belongs to the candidate keys, which suggests that the table satisfies the 3NF. The reason why we choose to keep 3NF rather than further decompose it to BCNF is the further decomposition will sacrifice the dependency preserving. There will be two tables, R1{`submission_date, season_id`} and R2{R − `season_id`}, after BCNF decomposition. Though the decomposition is lossless due to the fact that `submission_date` is the super key of R1, it will discard `customer_id, season_id` → `subscription_id`. Therefore, we choose to keep as 3NF.
Lossless join and dependency preservation	Lossless: The discarded decomposition is lossless. Preserving: The discarded decomposition is not dependency preserving.

`delivery`

Functional dependencies	• `subscription_id, week_no` → `status` (primary key)
Candidate keys	• `subscription_id, week_no` ({`subscription_id, week_no`}$^+$ = all attributes; chosen as primary key)
Normal form	`subscription_id` and `week_no` form the primary key of `delivery`. Therefore, the `delivery` satisfies BCNF.
Lossless join and dependency preservation	Lossless: No decomposition needed, not applicable. Preserving: No decomposition needed, not applicable.

refund

Functional dependencies	• refund_id → subscription_id • refund_id → amount • refund_id → type • refund_id → refund_date • refund_id → week_no • subscription_id, week_no → refund_id • subscription_id, type, refund_date → refund_id
Candidate keys	• refund_id • subscription_id, type, refund_date (for each subscription, there would be only one refund of specific type on a specific refund date)
Normal form	Satisfies BCNF, because the left-hand side of every non-trivial functional dependency is a super key: refund_id is a candidate key, and {subscription_id, type, refund_date} is also a candidate key. The second candidate key holds because the skipping windows of different deliveries do not overlap and a delivery can be skipped only once, a failed refund is issued on the day after its unique delivery day, and a subscription can be cancelled at most once. In the dependency subscription_id, week_no → refund_id, the subscription and week_no are the super key of the refund, which can not be primary key because the week_no can be null.
Lossless join and dependency preservation	Lossless: No decomposition needed, not applicable. Preserving: No decomposition needed, not applicable.

product

Functional dependencies	• product_id → season_id, name, is_farmers_choice, price_per_kg (primary key) • season_id, name → product_id, is_farmers_choice, price_per_kg
Candidate keys	• product_id ($\{\text{product_id}\}^+$ = all attributes; only key) • season_id, name
Normal form	The product_id is a super key of product. Since it is assumed that there will not be products in an identical name in a season, the season_id and name are also the super key of product. Therefore, product satisfies BCNF.
Lossless join and dependency preservation	Lossless: No decomposition needed, not applicable. Preserving: No decomposition needed, not applicable.

chosen_products

Functional dependencies	Only trivial dependencies.
Candidate keys	• subscription_id, product_id
Normal form	Satisfies BCNF, because the table only has trivial functional dependencies. Every attribute belongs to the only candidate key {subscription_id, product_id}, so there is no non-trivial functional dependency whose left-hand side is not a super key.
Lossless join and dependency preservation	Lossless: No decomposition needed, not applicable. Preserving: No decomposition needed, not applicable.

add_on

Functional dependencies	• `subscription_id, week_no, product_id` → `weight` (primary key) No other FD holds: • {`subscription_id, week_no`} does NOT determine `weight` (one delivery can carry several add-on products, each with its own weight) • `product_id` does NOT determine `weight` (the same product is weighed differently on different orders) • `weight` does NOT determine anything
Candidate keys	• `subscription_id, week_no, product_id` ({`subscription_id, week_no, product_id`}$^+$ = all attributes)
Normal form	The pair {`subscription_id, week_no`} is a composite FOREIGN KEY to `delivery`, but it is not a determinant inside `add_on` — so there is no partial dependency on the key. The sole determinant is the whole key, a superkey. `add_on` is in BCNF.
Lossless join and dependency preservation	Lossless: No decomposition needed, not applicable. Preserving: No decomposition needed, not applicable.

7 Traceability Matrix

Table 13 links every requirement to its ER construct, relations and constraints, enforcement and planned Stage 2 tests. Queries Q01–Q05 are checked against hand-computed results on the seed data, and all tests use a fixed reference date. Every rule enforced only by a SQL check has one limitation: a direct statement that bypasses our operations is not blocked.

Table 13. Traceability from requirements to design, enforcement and planned tests

Req.	ER construct	Relation / constraint	Enforcement	Planned test
Business rules				
BR1	*belong to*: subscription 1...1, customer 0...*, total for subscription; *schedule*: subscription 1...1, season 0...*; customer.email {CK1}	subscription.customer_id, season_id NOT NULL FK; uq_customer_email	DB constraint	A subscription with a missing customer or season and a duplicate email are rejected, and a valid subscription is accepted.
BR2	Key {customer, season} of subscription (Additional Information)	UNIQUE (customer_id, season_id)	DB constraint	A second subscription for the same customer and season is rejected, but one for the next season is accepted.
BR3	season {CK1} (season_name, year), {CK2} first_planning_day; derived start_day(), end_day(); subscription.status	season: two UNIQUE, CHECK season_name, CHECK DAYOFWEEK = 2; subscription: CHECK status	DB constraint; SQL check (status UPDATE at the reference date); Limitation (no scheduler)	Invalid season data and status values are rejected, and the status changes exactly on the start and end days.
BR4	*schedule*; subscription.submission_date, allocated_cost; renewal read from the customer's other subscription	Enrolment INSERT ... SELECT tests the window and the previous-season status	SQL check; Limitation	A submission outside the enrolment window writes no row, and renewing and new customers get the correct cost.
BR5	subscription.plan_type, frequency, allocated_cost; derived fee()	CHECK plan_type, CHECK frequency, ck_subscription_cost	DB constraint; Limitation (direct UPDATE)	Undefined plan types or frequencies and wrong costs are rejected, and each plan type with its correct cost is accepted .

Continued on next page

Table 13 (continued)

Req.	ER construct	Relation / constraint	Enforcement	Planned test
BR6	*grow*: season 10...10, product 1...1; *choose* M:N: subscription 0...8, product 0...*; product.is_farmers_choice	product.season_id NOT NULL FK; UNIQUE (season_id, name); chosen_products PK; count = 8, same season, change before range	DB constraint; SQL check; Limitation (10 and 6 by seed data)	Duplicate product names, a wrong number of products, products from another season and late changes are rejected, and a valid choice of 8 products commits.
BR7	Weak entity delivery; identifying *have*: subscription 0...12, delivery 1...1; partial key week_no; derived delivery_date(), add_on_fee(), price()	delivery PK (subscription_id, week_no), FK; CHECK week_no, status; generation INSERT ... SELECT; UPDATE ... WHERE status = planned; failed and refund in one transaction	DB constraint; SQL check	Deliveries are generated correctly, invalid weeks and invalid failures are rejected, and a failed status and its refund commit or roll back together.
BR8	subscription.remaining_ refundable_skip; delivery.status; *compensate*	CHECK BETWEEN 0 AND 2; skip transaction (status, window, refund, count)	DB constraint; SQL check	Late skips and counter values outside 0–2 are rejected, and a skip updates the status, refund and counter together.
BR9	*add on* M:N: delivery 0...8, product 0...*; attribute weight	add_on PK, composite FK, FK product; CHECK weight; guarded INSERT ... SELECT	DB constraint; SQL check	Non-positive weights, duplicate add-ons and add-ons to non-planned deliveries are rejected, and a valid add-on is priced correctly.
BR10	subscription.status, cancellation_date; *refunded via*; Additional Information note 3	Ordered cancellation transaction; UPDATE ... WHERE status IN (planned, active)	SQL check	Cancellation works before and during the season with the correct refund, a repeated cancellation writes nothing, and a failed step rolls back everything .
BR11	refund; *refunded via*: refund 1...1, subscription 0...*; *compensate* 1:1 optional; Additional Information notes 1–3	refund.subscription_id NOT NULL FK; composite FK to delivery; UNIQUE (subscription_id, week_no); CHECK type; ck_refund_link	DB constraint	Refunds with an undefined type, a wrong delivery link, a duplicate delivery or a non-existent delivery are rejected.
Use cases				
UC1	*belong to*, *schedule*, *choose*	subscription, chosen_products, season, product	Transaction (BR4, BR6 checks)	A full enrolment commits and an invalid one writes nothing.
UC2	*have*, *add on*, *compensate*, *refunded via*	delivery, add_on, product, refund	Transaction (guarded UPDATE + INSERT)	A failed delivery and its refund are saved together or not at all .
UC3	delivery, *compensate*, remaining_refundable_skip	delivery, subscription, add_on, refund	Transaction	A skip is accepted only before the deadline and refunded only while the counter allows it.
UC4	subscription, *have*, *refunded via*	subscription, delivery, add_on, refund	Transaction (ordered steps)	A cancellation commits once with the correct refund and rolls back fully on error.
Management questions				
MQ1	*schedule*, *refunded via*, *add on*	season, subscription, add_on, product, refund	Query Q01	Q01 matches the hand-computed net income for both plan types.
MQ2	*have*, *add on*	season, subscription, delivery, add_on, product	Query Q02	Q02 excludes add-ons on skipped and failed deliveries.

Continued on next page

Table 13 (continued)

Req.	ER construct	Relation / constraint	Enforcement	Planned test
MQ3	*have*	season, subscription, delivery	Query Q03	Q03 gives NULL for a week with only skips and ignores cancelled deliveries.
MQ4	*grow*, *choose*	season, product, chosen_products, subscription	Query Q04	Q04 lists a product chosen by nobody with 0.
MQ5	*schedule*, *belong to*	season, subscription (self-join)	Query Q05	Q05 returns only the plan type below 60%.

8 Work Plan and Risks

8.1 Division of Work

Table 14. Division of work

Task	Lead	Reviewer	Target date
Stage 1			
Scenario fingerprint and scope	SHI Jingxuan	CHEN Guanda	12 Oct 2026
Business rules, use cases and management questions	CHEN Guanda	SHI Jingxuan	16 Oct 2026
ER diagram	CHEN Guanda	SHI Jingxuan	19 Oct 2026
Relational mapping and data dictionary	SHI Jingxuan	CHEN Guanda	21 Oct 2026
Functional dependencies and normalization	CHEN Guanda	SHI Jingxuan	23 Oct 2026
Traceability matrix	Both	–	24 Oct 2026
Proposal assembly, metadata and AI declaration	CHEN Guanda	SHI Jingxuan	26 Oct 2026
Stage 2 (planned)			
Schema, constraints and index	SHI Jingxuan	CHEN Guanda	6 Nov 2026
Synthetic seed data and business queries (Q01–Q08)	CHEN Guanda	SHI Jingxuan	13 Nov 2026
View, transaction and tests (POS, N, TXC, TXR)	SHI Jingxuan	CHEN Guanda	18 Nov 2026
README, report and presentation	Both	–	23 Nov 2026

Every artefact has a lead and a reviewer from the other member. The reviewer checks it against the business rules and the traceability matrix, and records the result in the meeting notes and in the commit history of our private Git repository. Before each submission both members run all prototype scripts on MySQL 8.4.11, and each member explains two randomly chosen rules end to end (ER construct, table, constraint and test) to the other, so that either member can present and modify any part of the design.

8.2 Implementation Risks

Table 15. Main implementation risks and mitigations

Risk	Impact	Mitigation
Cross-row and cross-table rules (BR4, BR6–BR10) cannot be CHECK constraints	A direct INSERT or UPDATE outside our operations can break a rule	All changes go through the conditional statements of UC1–UC4 inside transactions; the gap is stated as a limitation; every SQL check has a “0 rows” or rollback test.
Date-dependent rules (windows, status, delivery days)	Results depend on the day the scripts run and cannot be reproduced	No `CURDATE()` or `NOW()`; every script sets a fixed reference date variable (e.g. `@ref_date = '2026-04-20'`) and compares with it.
Status is not updated automatically	A stale status gives a wrong renewal price (BR4) or the wrong cancellation case (BR10)	A status-update script for the reference date runs before every operation and demo; boundary test POS03.
Errors in refund amounts	Wrong net income in MQ1 and wrong refund tests	One shared expression for the price (allocated cost + add-on fee); hand-computed expected values in TXC02–TXC04 and POS15.
DDL failures: overlapping composite FKs, CHECK on an FK column, FK type mismatch	The Stage 2 schema does not load	Prototype DDL on MySQL 8.4.11 now; identical FK types; no referential actions; errors recorded in the meeting notes.
Seed data does not cover edge cases	HAVING, NULL handling and boundary tests show nothing	Seed data are derived from the test list: one plan type above and one below 60%, a week with only skips, a product chosen by nobody, renewal and non-renewal, a cancelled subscription.
Two-person team	One member cannot explain a part in the presentation	Lead/reviewer pairs, weekly walk-through, both members run every script.

References

[1] rimwood Farm. (n.d.). *Join the Brimwood Farm CSA*. Retrieved October 20, 2026, from https://www.brimwoodfarm.com/csa

[2] arcia, W. (2020, October 2). How to use CHECK constraint in MySQL 8. *Percona Blog*. https://www.percona.com/blog/how-to-use-check-constraint-in-mysql-8/

[3] Graph Ltd. (n.d.). *draw.io* [Computer software]. https://app.diagrams.net

[4] .R. Organics Farm. (n.d.). *CSA member handbook*. Retrieved October 20, 2026, from https://www.jrorganicsfarm.com/handbook

[5] ed Truck Farm. (n.d.). *Community supported agriculture*. Retrieved October 20, 2026, from https://amber-baker-83an.squarespace.com/community-supported-agriculture